

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΠΛΗΡΟΦΟΡΙΚΗΣ**



Τομέας: Υπολογιστική Υψηλών Επιδόσεων
Εργαστήριο: Εργαστήριο Πληροφοριακών Συστημάτων
Υψηλών Επιδόσεων

**Συγκριτική Ανάλυση και Πειραματική
Αξιολόγηση Συστημάτων Παράλληλου
Προγραμματισμού Βασισμένου σε
Εργασίες**

Διπλωματική Εργασία

του φοιτητή

Γεωργίου Τσάβου του Κωνσταντίνου

Αριθμός Μητρώου: 1084606

Επιβλέπων: Παναγιώτης Χατζηδούκας, Αναπληρωτής Καθηγητής,
Πανεπιστήμιο Πατρών

Πάτρα, Ιούνιος 2026

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα:

**Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες**

του φοιτητή του Τμήματος Μηχανικών Ηλεκτρονικών Υπολογιστών και
Πληροφορικής

Γεωργίου Τσάβου του Κωνσταντίνου

(Α.Μ.: 1084606)

παρουσιάστηκε δημόσια και εξετάστηκε επιτυχώς από την
τριμελή εξεταστική επιτροπή στις __/__/____.

Ο Επιβλέπων

Μέλος της Επιτροπής

Μέλος της Επιτροπής

Παναγιώτης
Χατζηδούκας
Αναπληρωτής Καθηγητής

Σπυρίδων
Σιούτας
Καθηγητής

Γεώργιος Παπαδημητρίου
Επίκουρος Καθηγητής

Στοιχεία Διπλωματικής Εργασίας

Θέμα:

Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες

Φοιτητής:

Γεώργιος Τσάβος του Κωνσταντίνου

Ομάδα Επίβλεψης:

Παναγιώτης Χατζηδούκας

Αναπληρωτής Καθηγητής, Πανεπιστήμιο Πατρών

Εργαστήριο:

Εργαστήριο Πληροφοριακών Συστημάτων Υψηλών Επιδόσεων

Περίληψη

Λέξεις-κλειδιά: HPC, Διαργασίες, task-parallelization, heterogeneity

Abstract

Keywords: HPC, Διεργασίες, task-parallelization, heterogeneity

Ευχαριστίες

Περιεχόμενα

Συντομογραφίες	ix
Κατάλογος σχημάτων	x
Κατάλογος πινάκων	xi
1 Εισαγωγή	1
1.1 Σκοπός και Αντικείμενο της Εργασίας	1
1.2 Δομή της Εργασίας	2
2 Θεωρητικό Υπόβαθρο	5
2.1 Παραλληλοποίηση με Εργασίες	5
2.2 Χρονοδρομολόγηση Εργασιών	6
2.3 Αρχιτεκτονικές NUMA	7
Βιβλιογραφία	9

Συντομογραφίες

HPC	High Performance Computing
FCFS	First-Come-First-Served
RR	Round Robin
NUMA	Non-Uniform Memory Access
HEFT	Heterogeneous Earliest Finish Time
GIL	Global Interpreter Lock

Κατάλογος σχημάτων

2.1	Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1])	. . .	6
-----	--	-------	---

Κατάλογος πινάκων

Κεφάλαιο 1

Εισαγωγή

1.1 Σκοπός και Αντικείμενο της Εργασίας

Η ραγδαία εξέλιξη της υπολογιστικής επιστήμης και η ανάγκη για την ανάλυση όλο και μεγαλύτερου όγκου δεδομένων έχουν καταστήσει τον παράλληλο προγραμματισμό αναπόσπαστο κομμάτι της σύγχρονης έρευνας και τεχνολογίας. Καθώς η φυσική αύξηση της συχνότητας των επεξεργαστών (ρολόι) έχει φτάσει στα όριά της, η σύγχρονη υπολογιστική ισχύς πηγάζει πλέον κυρίως από αρχιτεκτονικές πολλαπλών πυρήνων (multi-core) και συστήματα συμμετρικής ή μη-συμμετρικής πολυεπεξεργασίας. Παράλληλα, γλώσσες υψηλού επιπέδου όπως η Python κυριαρχούν στους επιστημονικούς υπολογισμούς λόγω της ευκολίας χρήσης τους και του πλούσιου οικοσυστήματος βιβλιοθηκών τους, όμως η αποτελεσματική εκμετάλλευση του σύγχρονου υλικού (hardware) μέσω αυτής αποτελεί μια μείζονα πρόκληση, κυρίως λόγω εγγενών περιορισμών της γλώσσας στον ταυτόχρονο προγραμματισμό.

Αντικείμενο της παρούσας διπλωματικής εργασίας είναι η συγκριτική ανάλυση και πειραματική αξιολόγηση δύο διαφορετικών συστημάτων παράλληλου προγραμματισμού βασισμένου σε εργασίες (task-based parallel programming). Το μοντέλο αυτό προσφέρει ένα υψηλό επίπεδο αφάιρησης, όπου το πρόγραμμα διασπάται σε ανεξάρτητες μονάδες εκτέλεσης, τις "εργασίες" (tasks). Ο προγραμματιστής ορίζει τις εργασίες και τις μεταξύ τους εξαρτήσεις, ενώ το σύστημα χρόνου εκτέλεσης (runtime system) αναλαμβάνει δυναμικά να τις δρομολογήσει στους διαθέσιμους πυρήνες, να διαχειριστεί τη μνήμη και να εξισορροπήσει τον φόρτο εργασίας, απαλλάσσοντας τον χρήστη από τον χειροκίνητο και επίπονο έλεγχο των νημάτων.

Σκοπός της εργασίας είναι να μελετηθεί πώς τα συστήματα αυτά ανταποκρίνονται στις σύγχρονες προκλήσεις του υλικού και του λογισμικού. Η πρώτη μεγάλη πρόκληση αφορά τις αρχιτεκτονικές **NUMA (Non-Uniform Memory Access)**, οι οποίες κυριαρχούν στα συστήματα υψηλών επιδόσεων (HPC). Σε αυτές, η μνήμη είναι χωρισμένη σε τοπικά τμήματα ανά επεξεργαστή, με αποτέλεσμα η πρόσβαση σε απομακρυσμένα τμήματα μνήμης να προκαλεί καθυστερήσεις και συμφόρηση. Τα task-based συστήματα πρέπει, επομένως, να έχουν επίγνωση της τοπολογίας (NUMA-awareness) για να διατηρούν την

τοπικότητα των δεδομένων. Η δεύτερη και σημαντικότερη πρόκληση στο περιβάλλον της Python είναι το **Global Interpreter Lock (GIL)**, ένας μηχανισμός που επιτρέπει την εκτέλεση κώδικα Python από ένα μόνο νήμα ανά πάσα στιγμή. Σε *cpu-bound* εφαρμογές, η χρήση απλών νημάτων οδηγεί σε *GIL thrashing* (διαρκείς εναλλαγές για την απόκτηση του lock), προκαλώντας κατακρήμνιση της χρήσης της CPU (ακόμα και στο 1.5%) και ακυρώνοντας τον παραλληλισμό.

Για την αντιμετώπιση αυτών των προβλημάτων, η εργασία εξετάζει και συγκρίνει δύο πλατφόρμες με θεμελιωδώς διαφορετικές αρχιτεκτονικές φιλοσοφίες:

- **StarPU (και η διεπαφή StarPUPY):** Ένα σύστημα χαμηλού επιπέδου σε C, σχεδιασμένο για *shared memory* και ετερογενή συστήματα. Θα επικεντρωθούμε στην Python διεπαφή του, StarPUPY για την πειραματική αξιολόγηση.
- **torcpy:** Ένα Python-native σύστημα βασισμένο στο κατανεμημένο πρότυπο MPI (μέσω της `mpi4py`), ιδανικό για κατενευμένα συστήματα.

Η πειραματική αξιολόγηση και η συνεισφορά της εργασίας εστιάζουν στην υλοποίηση και σύγκριση των δύο συστημάτων πάνω σε τρεις αντιπροσωπευτικές *cpu-bound* εφαρμογές: έναν αλγόριθμο βελτιστοποίησης (CMA-ES με επίλυση Poisson PDE), μια ροή επεξεργασίας εικόνας (Gabor Filters), και μια προσομοίωση Monte Carlo (Basket Options). Μέσα από αυτές τις εφαρμογές, η εργασία ποσοτικοποιεί τον χρόνο εκτέλεσης σε συνθήκες ισχυρού παραλληλισμού (64 πυρήνες), αναδεικνύοντας τα όρια και τις βέλτιστες πρακτικές χρήσης του κάθε συστήματος.

1.2 Δομή της Εργασίας

Η παρούσα διπλωματική εργασία είναι οργανωμένη σε επτά συνολικά κεφάλαια, ακολουθώντας μια λογική πορεία από τη θεωρητική θεμελίωση στην πρακτική εφαρμογή και την ανάλυση:

- **Κεφάλαιο 2 -- Θεωρητικό Υπόβαθρο:** Παρουσιάζεται το θεωρητικό υπόβαθρο γύρω από την παραλληλοποίηση με εργασίες, την ετερογένεια και τις προκλήσεις των αρχιτεκτονικών NUMA, την παραλληλοποίηση σε Python και το GIL.
- **Κεφάλαιο 3 -- Το Σύστημα StarPU:** Αναλύονται οι δυνατότητες και η αρχιτεκτονική του StarPU σε περιβάλλον C, καθώς και ο τρόπος διαχείρισης των πόρων.
- **Κεφάλαιο 4 -- Η Διεπαφή StarPUPY:** Εξετάζεται η μεταφορά του StarPU στην Python, οι μέθοδοι διαχείρισης του GIL και οι ιδιαιτερότητες του χρονοπρογραμματισμού του.

- **Κεφάλαιο 5 -- Το Σύστημα torcpy:** Αναλύεται το μοντέλο εκτέλεσης του torcpy, το API του, ο τρόπος δρομολόγησης των tasks και η χρήση του MPI για την απομόνωση διεργασιών.
- **Κεφάλαιο 6 -- Εφαρμογές και Μεθοδολογία:** Περιγράφεται αναλυτικά ο κώδικας και η μαθηματική δομή των τριών εφαρμογών (CMA-ES, Image Processing, Monte Carlo), καθώς και ο τρόπος λήψης των μετρήσεων.
- **Κεφάλαιο 7 -- Πειραματική Αξιολόγηση και Συμπεράσματα:** Παρουσιάζονται τα διαγράμματα και τα αποτελέσματα των μετρήσεων, αναλύονται τα φαινόμενα saturation και κλιμάκωσης, και συνοψίζονται τα τελικά συμπεράσματα της σύγκρισης.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

2.1 Παραλληλοποίηση με Εργασίες

Στον χώρο του HPC, η παραλληλοποίηση με εργασίες (task parallelism) είναι μια προσέγγιση που επιτρέπει την εκτέλεση πολλαπλών εργασιών ταυτόχρονα, όπου κάθε εργασία μπορεί να είναι ανεξάρτητη ή να έχει εξαρτήσεις από άλλες εργασίες. Αυτή η μέθοδος είναι ιδιαίτερα χρήσιμη όταν οι εργασίες έχουν διαφορετικά μεγέθη ή απαιτούν διαφορετικούς πόρους. Αυτό έχει ως αποτέλεσμα την καλύτερη εκμετάλλευση των διαθέσιμων πόρων του συστήματος και τη μείωση του χρόνου εκτέλεσης.

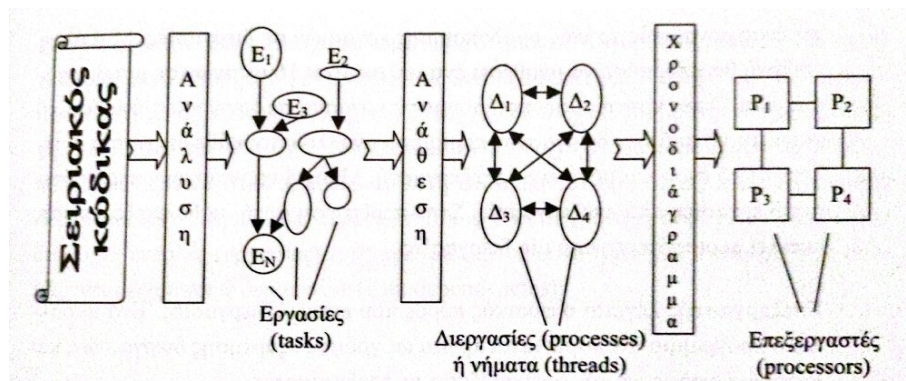
Η μετατροπή ενός αλγορίθμου για εκτέλεση σε ένα περιβάλλον παραλληλοποίησης με εργασίες απαιτεί διάφορα βήματα. Αρχικά, η υλοποίηση και η μελέτη του αλγορίθμου σε σειριακή μορφή σε μια γλώσσα προγραμματισμού θα βοηθήσει στην κατανόηση της λειτουργίας του και στην αναγνώριση των τμημάτων που μπορούν να παραλληλοποιηθούν. Ιδανικά αυτό θα θέλαμε να γίνεται αυτόματα από τον μεταγλωτιστή και το λειτουργικό σύστημα, αλλά στην πράξη απαιτείται η παρέμβαση του προγραμματιστή για να καθορίσει τις εργασίες και τις εξαρτήσεις τους [1].

Τα βήματα που πρέπει να λάβει υπόψη ο προγραμματιστής στην διαδικασία παραλληλοποίησης ενός αλγορίθμου, είναι τα εξής [1]:

1. **Ανάλυση του Αλγορίθμου σε Εργασίες (tasks):** Ο αλγόριθμος πρέπει να αναλυθεί για να εντοπιστούν τα τμήματα που μπορούν να εκτελεστούν ανεξάρτητα. Αυτό μπορεί να περιλαμβάνει την ανάλυση των βρόχων, των συναρτήσεων και των δεδομένων που χρησιμοποιούνται.
2. **Ανάθεση σε Διεργασίες (processes) ή Νήματα (threads):** Οι εργασίες πρέπει να ανατεθούν σε διαφορετικές διεργασίες (ή νήματα) ή να εκτελεστούν σε διαφορετικούς πόρους, με βάση την διαθεσιμότητα και τις απαιτήσεις του κάθε ενός.
3. **Χρονοδρομολόγηση των Διεργασιών (scheduling):** Η χρονοδρομολόγηση των εργασιών είναι ένα σύνθετο βήμα που αναλύεται στα εξής:

- (α') **Ανάθεση κάθε Διεργασίας σε ένα Πόρο:** Κάθε διεργασία πρέπει να ανατεθεί σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων.
- (β') **Απόφαση για την Εκτέλεση των Εργασιών:** Δηλαδή, πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε στο βήμα 3α'.

Τα παραπάνω βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες φαίνονται και στο παρακάτω Σχήμα 2.1.



Σχήμα 2.1: Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1])

2.2 Χρονοδρομολόγηση Εργασιών

Όπως είδαμε στην προηγούμενη ενότητα, η χρονοδρομολόγηση των εργασιών είναι το πιο σύνθετο βήμα από τα υπόλοιπα, το οποίο στην σημερινή εποχή γίνεται όλο και πιο δύσκολο λόγω της αυξανόμενης πολυπλοκότητας των συστημάτων HPC και των εφαρμογών που εκτελούνται σε αυτά.

Η χρονοδρομολόγηση των εργασιών περιλαμβάνει την ανάθεση κάθε διεργασίας σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων. Επιπλέον, πρέπει να ληφθεί υπόψη η απόφαση για το πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε. Αυτό καθιστά την επιλογή μιας αποτελεσματικής στρατηγικής (ή πολιτικής) χρονοδρομολόγησης κρίσιμη για την απόδοση των εφαρμογών σε περιβάλλοντα HPC.

Παλαιότερα ήταν αρκετό να γίνεται η χρήση απλών πολιτικών χρονοδρομολόγησης, όπως η First-Come-First-Served (FCFS) ή η Round Robin (RR), για να επιτευχθεί ικανοποιητική απόδοση. Ωστόσο, με την αύξηση της πολυπλοκότητας των συστημάτων και των εφαρμογών, αυτές οι απλές πολιτικές δεν είναι πλέον επαρκείς. Σήμερα, απαιτούνται πιο εξελιγμένες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη παράγοντες όπως οι

εξαρτήσεις μεταξύ των εργασιών, η εκτίμηση του χρόνου εκτέλεσης κάθε εργασίας, η δυναμική κατάσταση των πόρων και η τοπικότητα των δεδομένων κάθε εργασίας στην μνήμη, διότι πλέον τα συστήματα είναι πολύπολκα εισάγοντας έννοιες ετερογένειας (heterogeneity) και πολλαπλών επεξεργαστηκών μονάδων, όπως οι αρχιτεκτονικές **NUMA**.

Οπότε θέλουμε η χρονοδρομολόγηση των εργασιών να γίνεται δυναμικά είτε από το λειτουργικό σύστημα είτε από το περιβάλλον εκτέλεσης (runtime environment) με βάση τις τρέχουσες συνθήκες του συστήματος και των εργασιών. Αυτό μπορεί να επιτευχθεί μέσω της χρήσης τεχνικών όπως **Work Stealing**, **HEFT** κ.α. που λαμβάνουν υπόψη τους διάφορους παράγοντες που αναφέρθηκαν προηγουμένως. Τέτοιες τεχνικές υλοποιούνται όπως θα δούμε σε παρακάτω κεφάλαια και από τη **torcpy** και τη **StarPU** [2, 3] στο επίπεδο του περιβάλλοντος εκτέλεσης.

2.3 Αρχιτεκτονικές NUMA

Βιβλιογραφία

- [1] Παπαδάκης Στυλιανός και Διαμαντάρας Κωνσταντίνος. Προγραμματισμός και Αρχιτεκτονική Συστημάτων Παράλληλης Επεξεργασίας. 1η έκδοση. Αθήνα: Εκδόσεις Κλειδάριθμος, 2012. ISBN: 978-960-461-446-2.
- [2] Panagiotis E Hadjidakas κ.ά. ``torcpy: Supporting task parallelism in Python". Στο: SoftwareX 12 (2020), σ. 100517.
- [3] Cédric Augonnet κ.ά. ``StarPU: a unified platform for task scheduling on heterogeneous multicore architectures". Στο: European Conference on Parallel Processing. Springer. 2009, σσ. 863–874.

